Student ID (IT number): ..... **Name:** ..... **Module code: IT4090**

L1 Introduction to Cloud Computing

Evolution of IT infrastructure. Mainframe / Mini computers ➔ Personal computer ➔ Client / Server computing ➔ Web-based enterprise applications ➔ Cloud / Mobile computing

The electricity analogy. Mid 1800s, factories made their own electricity (dynamos and generators). Problems: **costly** to buy, install and maintain, **skilled engineers** needed, **spare parts** needed, **higher operating cost**, and it was **not the core business** or core skill. **Late 1800s, electricity providers** sent power over wires. Advantages: **no machines to buy or maintain**, **no specialised engineers**, **just connect to the grid and consume**, **no upfront payment (pay for what you consume)**, **anyone can connect easily**. Cloud is the same idea for IT: no own **data center**, consume resources as you wish, **pay only for what you consume**, focus on the **core business**.

Traditional IT process. Launching something like an e-commerce website the old way takes **6 to 12 months**: the decision stage (select vendors, proof of concept and evaluation, architecture, negotiations) takes 1 to 2 months, issuing the purchase order (hardware shipment, licences, contracts and SLAs) takes 2 to 3 months, hardware implementation (data center preparation, cabling and rack mounting, hardware installation) takes 1 to 2 months, and software implementation (installing software, testing, deploying, user acceptance testing) takes 2 to 4 months, and only then is the system live.

Inefficiencies of traditional IT. Capacity is a **fixed cost** bought from a forecast. When the actual load is above capacity there is **under-supply** and customers are lost, and when the load is below it there is **waste** of paid-for capacity. Slow, costly buying is also a **barrier for innovation**.

Cloud, the better alternative. 1 Provision servers, networks, storage, databases and applications in **minutes**, **2 pay-as-you-go (PAYG)** pricing, **3 add or remove capacity** as and when required, **4 destroy** the environment and stop paying. The capacity follows the actual load, so there is no under-supply, less over-supply (waste), a smaller initial investment, and capacity can be reduced when the load falls.

Why we need cloud. Business agility (faster time to market, more innovation, flexibility and scalability), **customer experience** (always available services, many venues or regions), **cost** (pay-as-you-go, **CAPEX to OPEX**, savings, focus on the core business).

NIST definition. Cloud computing is a model for enabling **ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources** (networks, servers, storage, applications and services) that can be **rapidly provisioned and released** with minimal management effort or service provider interaction (National Institute of Standards and Technology).

Five essential characteristics. On-demand self-service: the consumer provisions virtual servers, network and storage whenever needed without talking to the provider. **Broad network access:** capabilities are available over the network through standard mechanisms used by all kinds of clients. **Resource pooling:** the provider pools its resources and assigns and reassigns them dynamically to demand (multi-tenant), the customer does not know the exact location but may choose it at a higher level such as Region or Country. **Rapid elasticity:** resources are provisioned and released quickly, and this can happen automatically in line with demand. **Measured service:** usage is metered, so resources are controlled and optimised automatically, charged on a **pay-per-use** basis, and can be monitored, controlled and reported.

Technology or model. Cloud is **not a technology**. It is a **business model** and a **delivery model**. The technology underneath it is **virtualization**.

Cloud service models. Who manages what: orange layers are managed by the **customer**, blue layers by the **provider**.



IaaS: the provider manages the physical infrastructure and the virtualization layer. The consumer provisions servers, databases and networks, creates its own platform and deploys applications. **Compute, networks, content delivery, storage, backup and recovery (AWS EC2, Azure VM).**

PaaS: the provider manages the infrastructure and the platform that hosts the applications. The consumer deploys its own or acquired applications and can change hosting settings, but the application must use the languages, libraries, services and tools the provider supports. **Business intelligence, database, integration, application development and testing, deployment (Elastic Beanstalk, Azure App Service, RDS).**

SaaS: the consumer uses the provider's application running in the cloud and manages neither the infrastructure nor the application, only limited user-specific settings. **Email and office productivity, CRM, collaboration, document management, ERP (Gmail, Microsoft 365, Salesforce).**

Actors in cloud computing (NIST). The **cloud consumer** maintains a business relationship with and uses services from providers. The **cloud provider** is responsible for making a service available. The **cloud auditor** independently assesses cloud services, operations, performance and security. The **cloud broker** manages the use, performance and delivery of services and negotiates between providers and consumers. The **cloud carrier** provides the connectivity and transport of services from providers to consumers.

Deployment models. A **public cloud** is open to the general public, operated by a provider on the provider's premises: it is cost-effective, scales almost without limit on demand, needs zero maintenance, has continuous uptime and is reached over the internet, but it is multi-tenant, has a lower level of data security and raises regulatory compliance questions. A **private cloud** is for one organisation (several business units may use it), owned and managed by the organisation, a third party or both, located on or off premises: it gives high data security, less risk, single tenancy and compliance and is reliable, but it is expensive and has management overhead. A **hybrid cloud** combines private and public clouds and lets data and applications move between them: it is secure, cost-effective, flexible and scalable and portable, with data transfer as the weak point. A **community cloud** serves one community with shared concerns such as security, regulation or compliance, for example **government clouds**.

Cloud service providers. There are close to 300 providers. The top six are **AWS, Microsoft Azure, Google Cloud Platform (GCP), IBM Cloud, Alibaba Cloud and Oracle Cloud Infrastructure (OCI)**. The top three share about 60 percent of the market: AWS about 33 percent (steady since 2017), Azure about 20 percent (up from 11 percent in 2017) and GCP about 7 percent. All others together serve less than 40 percent.

L2 Cloud Core Services and Global Infrastructure

Cloud global infrastructure. It is designed and built to deliver a **flexible, reliable, scalable and secure** cloud computing environment with high-quality global network performance. It is the collection of hardware and software resources that make up the cloud: providers maintain **global data centers** with thousands of IT infrastructure components such as servers, physical storage devices and networking equipment.

Benefits of the global infrastructure. Security, availability, performance, scalability and flexibility.

Region, Availability Zone and data center. The cloud global infrastructure is built in three levels: a **Region** (a geographic area of the world) contains several **Availability Zones**, and each Availability Zone is made of one or more **data centers**. So the order is Region, then Availability Zone, then data center.

Selecting a Region. Choose the right Region for your services, applications and data using four factors: **data governance and legal requirements** (some data may have to stay in a country), **proximity to customers** (the closer the Region, the lower the latency), **services available within the Region** (not every service is offered in every Region), and **costs** (prices vary by Region).

Availability Zones. Each Region has **multiple Availability Zones**. Each Availability Zone is a **fully isolated partition** of the global infrastructure. An Availability Zone consists of **discrete data centers** and is interconnected with the other Availability Zones by **high-speed private networking**. **You choose** your Availability Zones. Because each zone is isolated, running in more than one zone keeps an application available when one zone fails.

Data centers. Data centers are where the **data resides and the data processing occurs**. Each data center has **redundant power, networking and connectivity** and is housed in a **separate facility**.

Core cloud services. The core services fall into three groups: **compute** (virtual servers, for example AWS EC2 and Azure Virtual Machines), **network** (virtual networks, for example AWS VPC and Azure Virtual Network) and **storage** (for example AWS S3 and EBS, Azure Blob storage and Disks). Every other service is built on these three.

AWS services (slide). AWS groups its services in three tiers. **Foundation services:** compute (Amazon EC2, AWS Lambda), storage and content delivery (Amazon CloudFront, Glacier, S3, EBS), database (DynamoDB, RDS, Redshift, ElastiCache) and networking (Route 53, VPC, Direct Connect). **Application services** (the middle tier): **administration and security** (Directory Service, IAM, Trusted Advisor, Config, CloudTrail, CloudWatch), **deployment and management** (CloudFormation, OpsWorks, CodeDeploy) and **analytics** (Kinesis, Data Pipeline, EMR). **Deployment and management services** (the top tier): application services (SQS, Elastic Transcoder, SES, AppStream, CloudSearch), mobile services (Cognito, Mobile Analytics, SNS) and enterprise applications (WorkDocs, WorkSpaces, WorkMail).

Azure services (slide). Azure shows its **platform services** in groups. Media and CDN (Media Services, Media Analytics, Content Delivery Network), integration (API Management, BizTalk Services, Logic Apps, Service Bus), compute services (Container Service, VM Scale Sets, Batch, RemoteApp, DevTest Labs), application platform (Web Apps, Mobile Apps, API Apps, Cloud Services, Service Fabric, Notification Hubs, Functions), developer services (Visual Studio, Mobile Engagement, VS Team Services, Xamarin, Application Insights, HockeyApp), data (SQL Database, SQL Data Warehouse, DocumentDB, SQL Server Stretch Database, Redis Cache, Storage Tables, Azure Search), intelligence (Cognitive Services, Bot Framework, Cortana) and analytics and IoT (HDInsight, Machine Learning, Stream Analytics, Data Catalog, Data Lake Analytics Service, Data Lake Store, IoT Hub, Event Hubs, Data Factory, Power BI Embedded).

Shared responsibility model. It is a framework that establishes **who is responsible for securing which aspects** of the cloud computing environment between the **cloud service provider (CSP)** and the **customer**. The CSP's responsibility is **security of the cloud** (the physical data centers, hardware, networking and virtualization layer). The customer's responsibility is **security in the cloud** (their data, access management, configuration, applications and, on IaaS, the operating system). The more managed the service, the more the provider takes over. This model is the same idea in AWS and in Azure.

AWS shared responsibility (slide). The customer is responsible for security in the cloud: customer data; platform, applications, identity and access management; operating system, network and firewall configuration; client-side data encryption, data integrity and authentication; server-side encryption (file system and/or data); and networking traffic protection (encryption, integrity, identity). **AWS is responsible for security of the cloud:** the software layer (compute, storage, database, networking) and the hardware / AWS global infrastructure (Regions, Availability Zones and Edge locations).

Azure shared responsibility (slide). The slide compares SaaS, PaaS, IaaS and on-premises. **Responsibility always retained by the customer** in every model: information and data, devices (mobile and PCs), and accounts and identities. **Responsibility varies by service type:** identity and directory infrastructure, applications, network controls and the operating system. For these the customer carries more as you move from SaaS to PaaS to IaaS and on-premises (the operating system, for example, is the provider's in SaaS and PaaS but the customer's in IaaS, while applications and network controls are the provider's in SaaS, shared in PaaS and the customer's in IaaS). **Responsibility transfers to the cloud provider:** the physical hosts, the physical network and the physical datacenter, which only an on-premises customer has to run itself.

How you pay for cloud services. Pay for what you use: pay only for the services you consume, with no large upfront expenses. **Pay less when you reserve:** invest in **Reserved Instances (RIs)**, committing for a period in return for a lower price. **Pay less when you use more:** realise **volume-based discounts**, so the unit price falls as usage grows.

Service limits and quotas. Every account has **default quotas**, and a quota is **specific to a Region**. **Soft limits** can be increased when you request it, **hard limits** cannot be increased. Limits exist to **limit potential excessive charges** and to **limit the overuse of resources by a single entity**.

L3 Cloud Compute and Storage, Part I (Compute)

Virtualization and containerization (recap). With **virtualization** each **virtual machine** contains the application, its dependencies and a full **guest OS**, and a **hypervisor** shares the hardware between the VMs. A **Type 1 hypervisor** runs directly on the hardware (bare metal), and a **Type 2 hypervisor** runs on top of a host OS. With **containerization** there is no guest OS: a **container** holds only the application (for example application XYZ) and its dependencies, many containers run on one **container engine** (Docker Engine) on the host OS, and they share the host OS kernel and common dependencies. That is why containers are lighter and start faster than VMs.

Software defined compute. Software-defined compute is when a **compute function is virtualized and abstracted from the hardware** it resides on. The computational functions of a data center can be **pooled across any number of processing units** and the workload can be dispersed among them. The hardware components tend to be **generic, industry-standard** parts, so they can be added easily to satisfy resource demand.

Cloud compute services. Amazon EC2 is AWS's offering of virtual machines in the cloud, provided as **IaaS**, started from pre-configured templates called **AMIs** (Amazon Machine Images), and billed **per second**. **Azure Virtual Machines** is Microsoft's offering of virtual machines in the cloud, also provided as **IaaS**, started from pre-configured templates from the **marketplace**, and billed **per minute**.

Instance types. The instance type decides the mix of CPU, memory, storage and network. **General purpose** balances compute, memory and network (web servers, small databases, dev and test). **Compute optimized** has high-performance CPUs (batch processing, gaming servers, scientific modelling, media encoding). **Memory optimized** has a lot of RAM (in-memory databases, real-time big data analytics). **Storage optimized** gives high disk I/O (data warehousing, NoSQL databases, log processing). **Accelerated computing** adds GPUs or other accelerators (machine learning, graphics rendering, video processing). **Burstable** instances run on a low baseline CPU and can burst above it when needed (websites with low or spiky traffic).

Tenancy. Default / shared: your instances run on physical hardware shared with other customers, which is the cheapest option. **Dedicated instances:** your instances run on hardware used only by your account, for isolation, but you do not control the physical host. **Dedicated hosts:** a whole physical server is allocated to you, so you control instance placement on it and can meet per-socket or per-core software licence rules, at the highest price.

Purchasing options. On-demand instances: pay for the compute capacity you use with no commitment, the most flexible and the highest price, good for short, unpredictable workloads. **Reserved instances:** commit to a period (1 or 3 years) for a big discount, good for steady, long-running workloads. **Spot instances:** use spare capacity at a very large discount, but the provider can take the instance back at any time, good for fault-tolerant jobs that can be interrupted. **Savings plan:** commit to a consistent amount of usage (dollars per hour) for 1 or 3 years for lower prices while staying flexible about instance type.

Compute instance lifecycle (Azure VM). A VM is **starting**, then **running**. From running, **stop** moves it to **stopping** and then **stopped** (still holding its compute resources). **Deallocate** moves a running or stopped VM to **deallocating** and then **deallocated**, which releases the compute resources. A deallocated VM can be **started** again (back to starting) or **powered off** to stopped, and a stopped VM can also be started again.

Compute instance lifecycle (AWS EC2). An instance is launched from an **AMI** and goes to **pending** and then **running**. A running instance can be **rebooted** (it goes through rebooting and returns to running), **stopped** (stopping, then stopped, and later **started** again through pending), or **terminated** (shutting-down, then terminated, which cannot be undone). Stop and start apply to **EBS-backed instances only**, and a stopped instance can also be terminated.

Placement groups. A placement group controls how instances are placed on the underlying hardware. In **AWS** there are three kinds: **cluster** placement groups (instances packed close together in one Availability Zone for low latency and high network throughput), **partition** placement groups (instances spread across logical partitions that do not share hardware, used for large distributed systems) and **spread** placement groups (each instance on separate hardware to reduce correlated failure). **Azure** has **proximity placement groups**, which place VMs physically close to each other to reduce latency.

L4 Cloud Compute and Storage, Part II (Storage)

Software defined storage (SDS). It is a storage architecture that **abstracts data storage resources from the underlying hardware**. Storage from many devices is managed as one pool by software, so capacity can be added easily and managed through policies.

CAP theorem. It is impossible for a distributed data store to provide **more than two out of the following three guarantees** at the same time. **Consistency:** every read receives the most recent write or an error (all clients see the same view of the data, even right after an update or delete). **Availability:** every request receives a non-error response, without the guarantee that it contains the most recent write (all clients can find a replica of the data, even when some nodes fail). **Partition tolerance:** the system continues to operate despite an arbitrary number of messages being dropped or delayed by the network between nodes (it works as expected even with a partial network failure). The overlaps of the three circles give the pairs **CA, CP and AP**, and all three together is impossible.

Consistency or availability. In the presence of a **network partition we must choose between consistency and availability**. When a network partition failure happens we can **cancel the operation**, which decreases the availability but ensures consistency, or we can **proceed with the operation**, which provides availability but risks inconsistency.

Types of storage. Cloud storage comes as **block storage, file storage and object storage**.

Block storage. It manages data as **blocks within sectors and tracks**, and each block has a **block ID**. A set of blocks can be organised as a **file system**. It is suitable for **transactional and structured data** (file systems, databases, logs) and is optimised for block-level performance, which is measured in **IOPS**. Examples: **AWS Elastic Block Store (EBS)** and **Azure Disks (Page Blob storage)**.

File storage. It manages data as a **file hierarchy** and is used over a **shared network**. The storage manages the underlying data on disk: the storage server uses block storage with a local file system. The protocols supported are **NFS (Network File System)** and **SMB (Server Message Block)**. Examples: **AWS Elastic File System (EFS)** and **Azure Files (File storage)**.

Object storage. It manages data as **objects**, and each object consists of the **data, metadata and a globally unique identifier**. Clients access it through **HTTPS, APIs or SDKs**. Examples: **AWS Simple Storage Service (S3)** and **Azure Blobs**.

Object storage features. Versioning: keeps earlier versions of an object. **Encryption:** client side (you encrypt before upload) or server side (the service encrypts), using **service managed keys, customer managed keys or customer provided keys**. **Immutability:** time-based retention (the object cannot be changed or deleted for a set time) and legal hold (it is locked until the hold is removed). **Web hosting:** a static website can be served directly from object storage.

Object storage tiers. CSPs offer different access tiers designed for different use cases. **AWS: Standard, Intelligent Tiering** (moves objects between tiers automatically), **Infrequent Access** and **Glacier** (archive). **Azure: Hot**, **Cold** and **Archive**. Data that is used often goes in the hot or standard tier, and data that is rarely used goes in cooler tiers that cost less to store but cost more or take longer to retrieve.

Other important aspects of cloud storage. Archiving and backup: keeping copies of data for long-term retention and recovery. **Hybrid storage:** connecting on-premises storage with cloud storage so that both are used together. **Bulk data transfer:** moving very large amounts of data to the cloud, using the network or physical transfer devices.

L5 Cloud Networking

Networking recap. Know **IP addresses, subnets / VLSM, basic routing and default gateways**. A subnet is a smaller part of a larger IP range.

Software defined networking (SDN). SDN is a **network management architecture** that enables **dynamic, programmable network configuration** to improve network performance and monitoring. It **decouples the network control functions from the forwarding functions**, so the network control becomes directly programmable and the underlying infrastructure is abstracted for applications and network services.

Network function virtualization (NFV). NFV is a network architecture that **decouples physical network functions (PNF) from proprietary hardware appliances** and runs them as software in virtual machines. These **virtual network functions (VNF)** include firewalls, traffic control, virtual routing and so on. **Cloud-native network functions (CNF)** are the successor of VNFs: a software implementation of a PNF that runs **inside a Linux container**. Advantages of CNF: speed, flexibility and efficiency, and they fit monitoring, auto scaling, security and deployment through DevOps and CI/CD.

SDN vs NFV. Similarities: both use network abstraction and both depend heavily on virtualization. **Differences:** SDN **separates the network control functions from the forwarding functions** and defines the big-picture side of the infrastructure, while NFV **abstracts all network functions from the underlying hardware** and delivers a wide range of specific functionalities.

Virtual networks. A virtual network is a cloud networking service that **provides networking functions to cloud resources**. It is **logically isolated** from other virtual networks, has **one or more IP ranges** and **one or more subnets**, and can **peer** with other virtual networks if the IP ranges **do not overlap**. Implementations: **AWS VPC (Virtual Private Cloud)** and **Azure Virtual Network**.

Subnets. A subnet **segments the virtual network IP range even further**. Subnet CIDR blocks **cannot overlap** within a virtual network, and traffic can be restricted using **access control lists**. **AWS** offers **public and private subnets**: instances in a public subnet can send outbound traffic directly to the internet, instances in a private subnet cannot. **Azure** does not have the public / private subnet concept: all subnets can be considered public, and private subnet behaviour is built with **route tables**.

Virtual network and subnet sizing. Allowed IP ranges are the **RFC 1918 private ranges**: 10.0.0.0/8 (10.0.0.0 to 10.255.255.255), 172.16.0.0/12 (172.16.0.0 to 172.31.255.255) and 192.168.0.0/16 (192.168.0.0 to 192.168.255.255). IPv4 block sizes allowed: **Azure /2 to /29**, **AWS /16 to /28**. **Five IPs in every subnet are reserved**: the 1st is the **network address**, the 2nd is reserved for the **router**, the 3rd is reserved for **DNS**, the 4th is reserved for **DNS / future use**, and the last is the **network broadcast address**. Number of usable IPs = $2^{\text{(32 minus prefix)}}$ **minus 5**.

Subnet IP reservation, worked examples. 10.0.0.0/24: network address 10.0.0.0, reserved 10.0.0.1, 10.0.0.2 and 10.0.0.3, broadcast 10.0.0.255, first usable 10.0.0.4, last usable 10.0.0.254, usable IPs 2^8 minus 5 = **251**. **172.16.10.128/25:** network address 172.16.10.128, reserved .129, .130, .131, broadcast 172.16.10.255, first usable 172.16.10.132, last usable 172.16.10.254, usable IPs 2^7 minus 5 = **123**. **192.168.0.64/27:** network address 192.168.0.64, reserved .65, .66, .67, broadcast 192.168.0.95, first usable 192.168.0.68, last usable 192.168.0.94, usable IPs 2^5 minus 5 = **27**.

Virtual network interfaces. A virtual network interface is a **logical networking component that represents a network interface card (NIC)**. It lets a compute instance communicate with other instances and / or the internet and is attached to a compute instance. More than one can be attached to the same instance, and an interface can have **one public IP and one or more private IPs**. Implementations: **AWS Elastic Network Interface (ENI)** and **Azure Network Interface Card (NIC)**.

Access controlling. It works as a simple firewall to **control inbound and outbound traffic** to a subnet or virtual network interface. A rule contains a **rule number, protocol, source / destination port ranges, source / destination IP ranges and an allow / deny action**. The usage varies by CSP: **AWS security groups** are assigned to an EC2 instance, the **AWS network access control list (NACL)** is assigned to a subnet, and **Azure network security groups (NSG)** can be assigned to both subnets and NICs.

Stateful and stateless rules. With **stateful** rules you only specify the rule for the request, with no need for a rule for the response. With **stateless** rules you must specify rules for the request as well as the response. **AWS security group: stateful. AWS NACL: stateless. Azure NSG: stateful.**

Network gateway services. These are virtual network functions that provide connectivity between cloud resources and outside networks or the internet: **internet gateway, NAT gateway, VPN gateway and network transit gateway**.

Internet gateway. It provides connectivity between the internet and the virtual network, **scales horizontally** and has **no bandwidth constraints**. Implementation: **AWS internet gateway**. Azure does not have the internet gateway concept: internet connectivity is provided by default through the default gateway.

NAT gateway. It provides **network address translation (NAT)** to compute instances in private subnets, so they can reach the internet, and it **prevents connections to the subnets from being initiated from the internet**. The NAT gateway must be deployed in a **public subnet**. Implementations: **AWS NAT instance, AWS NAT gateway, Azure Virtual Network NAT**. In the AWS diagram an instance in the private subnet sends traffic to the NAT gateway in the public subnet, which sends it to the internet gateway and on to services such as S3, while an instance in the public subnet uses the internet gateway directly.

VPN gateway. VPN gateways create **encrypted tunnels over the public internet** to send encrypted traffic between virtual networks and on-premises networks. Implementations: **Azure VPN Gateway** and **AWS Virtual Private Gateway**. Two VPN types: **site-to-site** (between a virtual network and an on-premises network, or between two virtual networks) and **point-to-site** (between a virtual network and a client PC running special client VPN software). Azure VPN Gateway provides both. AWS Virtual Private Gateway provides only site-to-site, and point-to-site comes from a separate service, **AWS Client VPN**.

Network transit gateway. It connects **multiple virtual networks and on-premises networks through a central hub**. This architecture used to be built as the **hub and spoke model**. Implementations: **AWS Transit Gateway** and **Azure Virtual WAN**. In the AWS diagram a corporate office or data center connects through AWS Direct Connect (point-to-point link, BGP) or an IPsec VPN tunnel to the transit gateway, which attaches to several VPCs (for example 10.1.0.0/16, 10.2.0.0/16 and 10.3.0.0/16) with one VPC attachment each.

Network peering. It allows two virtual networks to **connect and communicate** with each other (it used to be done through VPN gateways). Each virtual network can be peered with more than one other, and peering works **across accounts / subscriptions**. It **does not support transitive peering**: if VNet A is peered with B and B is peered with C, A cannot communicate with C and the reverse. Both AWS (VPC peering) and Azure (VNet peering) have it.

Bastion hosts. A bastion host, also called a **jump host**, is a special compute instance deployed to give **secure management access to other compute instances in private subnets**, without exposing those instances to the internet. The bastion host must be placed in a **public subnet** and given a **public IP**. Azure also offers the Azure Bastion host service.

Hybrid connectivity. To **connect remote users** to a virtual network in the CSP: direct internet connectivity, through a VPN with **Azure point-to-site VPN** or **AWS Client VPN**. To **connect remote networks: site-to-site VPN** or **direct connectivity** with **Azure ExpressRoute** or **AWS Direct Connect**. Direct connectivity is a private dedicated connection that does not use the public internet. In the AWS diagram the Direct Connect location links the AWS Direct Connect endpoint to the customer or partner router, with a **public VIF** reaching public services (Lambda, DynamoDB, S3) and **private VIFs** reaching VPCs through virtual private gateways, over the service provider network to the customer premises.

Private access to cloud services. Private links provide access to the CSP's **PaaS services from the virtual network without going through the public internet**. **Private endpoints** are connected to a virtual network with an IP and give other resources in the virtual network access to PaaS services through the private link (Azure Private Link, AWS PrivateLink). **Azure service endpoints** provide secure, direct connectivity to Azure PaaS services over an optimised route on the **Azure backbone network**, but the traffic from the virtual network **still hits the public endpoint of the PaaS service**. In the diagram the VM reaches Azure SQL through the service endpoint, the VNet firewall rules apply, and the database sees the VM's private IP (10.1.0.2) as the source.

Network flow logs. They capture **information about IP traffic going in and out of network interfaces** in the virtual network. The log data is collected **outside of the network traffic path**, so it does **not affect network throughput or latency**. Flow log data can be stored in a separate object storage (AWS S3 / Azure Blobs). **Azure flow logs** are captured from NSGs, and **AWS flow logs** can be created for VPCs, subnets and / or ENIs. Use cases: **troubleshooting connectivity issues, intrusion / anomaly detection, and archival for compliance, regulatory and legal purposes**.

L6 Cloud High Availability and Disaster Recovery

Downtime. Downtime is a **period when a system is unavailable (offline)**. **Planned / scheduled downtime** is the result of scheduled maintenance, patch installation or a reboot. **Unplanned / unscheduled downtime** is the result of a hardware, software or network failure, a security breach or a physical event.

Availability and the nines. Availability % decides the allowed downtime: **downtime = (100 minus availability %) / 100 x the period**. Each extra nine cuts the downtime by about **ten times**. Per year: **99% (two nines) = 3.65 days**, **99.9% (three nines) = 8.77 hours**, **99.99% (four nines) = 52.6 minutes**, **99.999% (five nines) = 5.26 minutes**.

High availability (HA). HA is a characteristic of a system which aims to **ensure an agreed level of operational performance, usually uptime, for a higher-than-normal period**. Three principles: **elimination of single points of failure** (a part of a system that, if it fails, will crash the entire system), **reliable crossover** (the ability to switch systems without losing data) and **detection of failures as they occur**.

Redundancy. Redundancy is the **duplication of critical components or functions of a system** with the intention of increasing the reliability of the system. **Passive redundancy** achieves high availability by including **enough excess capacity in the design to accommodate a performance decline**. **Active redundancy** achieves high availability using **complex systems that detect failure and automatically reconfigure the system to bypass failed items, using a voting scheme**.

Failover and failback. Failover is switching to a **redundant / standby / passive instance upon system failure**, and it uses a **heartbeat signal** to detect failures. **Failback** is the process of **restoring a failed system back to its original working state**.

Replication. Replication is **sharing information so as to ensure consistency between redundant resources**, such as software or hardware components, to improve reliability, fault tolerance or accessibility. **Active replication:** the **same request is processed at every replica**. **Passive replication: all requests are processed in one replica and the result is transferred to the others**.

High availability clusters. An HA cluster is a **group of computers that support applications and can provide continued service when system components fail**. The areas to make reliable are **node reliability, storage** (disk mirroring, redundant SANs), **network and electricity**. Node configurations: **Active/Active** (all nodes serve requests at the same time) and **Active/Passive** (a standby node waits and takes over on failure).

Load balancing. It is the process of **efficiently distributing network traffic across a group of backend servers**. **Session persistence** means all requests from a client are sent to the **same backend server for the duration of the session**. Algorithms: **Round Robin** (requests go to the servers in turn), **Least connections** (to the server with the fewest active connections), **Least time** (to the server with the fastest response) and **IP Hash** (the client IP decides the server, so the same client reaches the same server).

Business continuity. Business continuity is the **capability of an organization to continue the delivery of products or services at pre-defined acceptable levels following a disruptive incident**. **Business continuity planning (BCP)** is the process of creating **systems of prevention and recovery to deal with potential threats to a company**. **BCP analysis: business impact analysis, business threat analysis and business impact scenarios**.

Disaster recovery (DR). DR is the **part of the BCP** which involves a set of **policies, tools and procedures to enable the recovery or continuation of vital technology infrastructure and systems** following a natural or human-induced disaster. Elements of a DR plan: **DR team, risk evaluation, business critical asset identification, backup strategy, and testing and optimization**.

Backup strategy. A backup is a **copy of computer data taken and stored elsewhere** so that it may be used to restore the original after a data loss event, and backing up is the process of taking a backup. Backup methods: **full backup** (a copy of all the data), **incremental** (only data changed since the last backup of any kind), **differential** (all data changed since the last full backup) and **near continuous data protection** (changes are captured almost continuously).

DR tiers. Tier 0: no off-site data. **Tier 1:** data backup, no hot site. **Tier 2:** data backup with hot site. **Tier 3:** electronic vaulting. **Tier 4:** point in time copies. **Tier 5:** transaction integrity. **Tier 6:** zero or little data loss. **Tier 7:** highly automated, business-integrated solution. A higher tier means faster recovery and less data loss, but a higher cost.

Recovery objectives. Recovery Time Objective (RTO): the targeted period in which a business process must be **restored after a disaster** (how long we can be down). **Recovery Point Objective (RPO):** the targeted period in which a **data loss is acceptable** due to a disaster (how much data we can lose). **Recovery Consistency Objective (RCO):** the amount of **data inconsistency** that an organization is willing to tolerate in a recovered data set.

Cloud computing link. The closing slide ties HA and DR to three cloud benefits. **Scalability:** the cloud adds or removes servers and capacity as demand changes, so extra nodes behind a load balancer handle more traffic. **Availability:** the system stays accessible and in service, achieved with redundancy across Availability Zones, failover and load balancing, measured by the nines. **Reliability:** the system works correctly and consistently without failing, achieved by removing single points of failure, replication, backups and a tested DR plan.

L7 Cloud Security and Monitoring

Software defined security (SDS). SDS is a type of security model in which the information security of a computing environment is **implemented, controlled and managed by security software**. It is **software-managed, policy-driven and governed security**, where most security controls such as **intrusion detection, network segmentation and access controls** are automated and monitored through software.

Principle of least privilege (PLOP). PLOP is an information security concept: a **user or a process should only be granted the privileges that are essential to perform its intended function**, and nothing more. This limits the damage if an account is misused or compromised.

Identity and access management (IAM). IAM **solves the principle of least privilege**. It is a **framework of policies and technologies** to ensure users and processes have the appropriate access to technology resources. IAM is used to **identify, authenticate and authorize** the users or processes that will use those resources.

IAM components: organization, users and groups. The **logical organization** is the logical boundaries and operating domains for a given user or process. **Users** are users, applications or services within the logical organization that need to access resources: each has its **own security credentials**, can be part of one or more

groups, and by default has **no permission to do anything**. **Groups** are a collection of users and do **not have security credentials**.

IAM components: policies and roles. Policies are a collection of permissions, and the **permissions determine whether a request is allowed or denied**. **Roles** are the main component of **role based access control (RBAC)** and are a collection of policies. Roles can be **assumed by users or services**, and when assumed they give **temporary security credentials** for the role session.

Multi factor authentication (MFA). MFA is an electronic authentication method in which a user is granted access only after successfully presenting **two or more pieces of evidence (factors)** to an authentication mechanism. The factors: **something the user has** (security token, smart card), **something the user knows** (password, PIN), **something the user is** (biometrics: fingerprint, face, voice, iris) and **somewhere the user is** (GPS location).

Federated identity and single sign on (SSO). A **federated identity** is an electronic identity **linked and stored across multiple identity management systems**. If we use multiple identities across multiple systems, it is tedious to maintain different user IDs and passwords. **SSO** lets users move between multiple systems **without logging in to each one, through one verified identity**. There are two types of identity federation: **enterprise identity federation** (the organization's own directory is trusted) and **web identity federation** (a public identity provider such as Google or Facebook is trusted).

Encryption recap. Client side vs server side encryption: the client encrypts before upload, or the cloud service encrypts after it receives the data. **Stages of data encryption: data in use** (being processed), **data in transit** (moving over the network) and **data at rest** (stored). **Types of encryption: symmetric key** (the same key encrypts and decrypts), **asymmetric key** (a public key encrypts and a private key decrypts) and **hashing** (a one-way function, it cannot be reversed).

Cloud security risks and threats. Poor access management, data breach / leak / loss, misconfiguration, insecure APIs, account hijacking, lack of visibility and DoS / DDoS attacks.

Monitoring in the cloud. Cloud monitoring is a method of **reviewing, observing and managing the operational workflow** in a cloud-based IT infrastructure. Manual or automated management techniques confirm the **availability and performance** of websites, servers, applications and other cloud infrastructure. Example services: **Datadog, AppDynamics, Azure Monitor and Amazon CloudWatch**.

Metrics, events and logs. Metrics are raw data about resource usage or behaviour, collected via a **monitoring agent** (for example CPU, memory and disk space usage). **Events** are generated by systems when something happens and capture **what happened, where and when** (for example events from automation tools). **Logs** provide information about **what systems have been doing** and are extremely valuable in troubleshooting.

Alerts. Alerts are the **reactive element** of the monitoring system and trigger actions based on changes in metrics, events or logs. Types: **threshold based alerts** (a value crosses a set limit), **anomaly detection based alerts** (behaviour differs from the normal pattern) and **heartbeat alerts** (the regular signal from a resource stops). Alert outputs: **notifications** and **automated actions**.

L8 Cloud Database and Analytics Services

Relational / SQL databases. Data is held in **highly structured table organization** with **rigidly-defined formats** and **dependencies among tables**. They **enforce ACID** (**Atomicity, Consistency, Isolation, Durability**), reduce anomalies, enforce integrity and use **SQL** to access data. Examples: **MS SQL, MySQL, Oracle, PostgreSQL, Amazon RDS**.

Non-relational / NoSQL databases. They are **document oriented**, handle **large and complex queries** and support **rapidly changing designs**. Examples: **MongoDB, Cassandra, CosmosDB, Redis, CouchDB, Aurora** (as listed on the slide).

Database workloads. OLTP (Online Transaction Processing): focus on **operational data** and transaction processing, **small, simple ad-hoc queries**, responses in **milliseconds**, **highly normalized**. **OLAP (Online Analytical Processing):** focus on **historical data**, data analysis and reporting, **large, complex queries**, **data warehouses**, response times from **seconds to hours**, typically **denormalized**.

Database models and workloads together. Relational / OLTP: relational databases (Oracle, PostgreSQL, MS SQL, MySQL). **Relational / OLAP:** relational analytics (the same products). **Non-relational / OLAP:** big-data analytics (Hadoop, HDInsight). **Non-relational / OLTP:** key-value, columnar and document stores (MongoDB, Cassandra, Riak).

Types of NoSQL databases. Key-value: indexed keys and values, use case **session data and shopping cart**. **Document store:** stores data in documents (XML, JSON), **schema less**, documents contain key-value pairs, use case **e-commerce and analytics**. **Columnar:** optimized to retrieve columns of data, use case **CMS and blogging platforms**. **Graph:** presents interconnected data as logical graphs, with the focus on **relationships**.

NoSQL comparison. Key-value store: flexibility high, complexity none, performance high, scalability high. **Column store:** flexibility moderate, complexity low, performance high, scalability high. **Document:** flexibility high, complexity low, performance high, scalability variable (high). **Graph DB:** flexibility high, complexity high, performance variable, scalability variable.

CAP theorem. The slide repeats the CAP theorem from L4: a distributed data store cannot provide more than two of **Consistency, Availability and Partition tolerance** at the same time, so in a network partition you choose consistency or availability.

Database caching. Caching is a **buffering technique that stores frequently requested data in temporary memory**. It facilitates data access and **reduces database workloads**. Two popular caching systems: **Redis** and **Memcached**.

Redis vs Memcached. Redis: open source, **in-memory key-value data store**, sub-millisecond response times, supports various data structures (strings, lists, sets and so on), **persistent** (the cache survives reboots), supports read replicas, atomic operations, backup / restore and HA. **Memcached:** open source, **in-memory object store**, sub-millisecond response times, supports strings and objects, **not persistent** (the cache does not survive reboots), supports **scaling out and multithreading**.

Data warehouse. A data warehouse is a type of data management system designed to **enable and support business intelligence (BI) activities, especially analytics**. It is intended for **querying and analysis only** and often contains **large amounts of historical data**. It is a repository for **structured, filtered data that has already been processed for a specific purpose**, typically derived from a wide variety of sources such as application log files and transaction applications. Two approaches: **ETL (Extract, Transform, Load)**, where data goes source, then staging, then destination, and **ELT (Extract, Load, Transform)**, where data goes source straight to destination and is transformed there.

Data lake. A data lake is a storage repository that holds a **large amount of raw data in its native format until it is needed**. While a hierarchical data warehouse stores data in files or folders, a data lake uses a **flat architecture** to store the data.

Data warehouse vs data lake. Data warehouse: processed data, data currently in use, used by **business professionals**. **Data lake: raw data**, purpose of the data not determined yet, used by **data scientists**.

Real time data processing. It is the **quickest data processing technique**, executing data in a short period of time and providing the most accurate output. The processing is done **as the data is inputted**, so it needs a **continuous stream of input data** to provide a continuous output. It is also known as **stream processing**.

L10 Cloud Migrations

Cloud migration. Cloud migration is the **movement of a meaningful portion of your organization's existing IT assets to the cloud**. The migration process has four phases: **opportunity evaluation, portfolio discovery and planning, designing / migrating / validating, and operating**. The strategy used for each application is chosen in the planning and design phases.

Opportunity evaluation. The question is: what is the **business case or the compelling event** that will drive your migration to the cloud? Typical drivers: **hardware recycle / renewal time approaches, data center lease expires, gain more productivity, global expansion, mergers and acquisitions, and the drive for standardized architectures**.

Portfolio discovery and planning. Ask **what is in your environment, what are the interdependencies, what will you migrate first and how will you migrate it**. Tools: **AWS Discovery Service, Azure Migrate Service** and third party cloud migration tools and services.

Designing, migrating and validating. Migrate **individual applications**, starting with the **least complex applications and moving towards more complex ones**, and design, migrate and validate each application according to its migration strategy. Steps: **test and decommission old applications / hardware, run parallel environments temporarily if needed, switch traffic, users and content partially**, and do the **full cutover to the cloud when everything is working as expected**.

Operating. Operating in the cloud requires a **modern operating model**: **DevOps, NoOps, FinOps and SecOps**. It is an **evergreen set of people, process and technology** that constantly improves as you migrate more applications. Use the first set of application migrations to build the foundation of the modern operating model and improve it continuously.

Cloud migration strategies. Gartner's 5 R's: Rehost, Refactor, Revise, Rebuild, Replace. **Azure's 4 R's:** Rehost, Refactor, Re-architect, Rebuild. **AWS's 6 R's: Rehost, Re-platform, Repurchase, Refactor / Re-architect, Retire, Retain**.

Rehost. Known as **lift-and-shift**. It does **not require application / code changes**: applications are migrated as-is to the cloud, then optimized or re-architected once they are in the cloud.

Re-platform. Known as **lift-tinker-and-shift**. It means migrating to a different platform **without changing the core architecture** of the application. Examples: migrating web sites from VMs to PaaS (Azure App Service, Amazon Elastic Beanstalk), migrating databases from VMs to PaaS (Amazon RDS, Azure SQL), changing the platform of databases (Oracle to PostgreSQL) and changing the platform of application logic (Oracle WebLogic to Apache Tomcat).

Repurchase. Moving to a different platform, mainly to a **SaaS platform**. Examples: a CRM to **Salesforce.com**, an HRIS to **Workday**, a CMS to **SharePoint**.

Refactor / Re-architect / Rebuild. It is **re-imagining how the application can be architected and developed using cloud native features**, to get the optimal usage of cloud native features and improve **application performance, scalability, addition of new features, agility and business continuity**. It typically includes migrating **monolithic applications to a service-oriented or serverless architecture**, and it tends to be the **most expensive** migration strategy.

Retire. Get rid of applications that are **no longer useful and can be turned off**. Benefits: more attention to important applications, reduced security risks and cost savings.

Retain. Revisit migrating these applications later, or do nothing for now. These are recently purchased or upgraded applications that cannot be migrated at this point, applications being sunset and low priority applications.

Cloud migration challenges. Cost of migration: focus on planning, follow an incremental adaptation process and use a hybrid cloud approach. **Resistance to adapt:** get leadership buy-in and choose intuitive tech solutions. **Shortage of skills:** invest in expert training and resources, build a culture of continuous learning and get help from vendors to skill up in-house resources.

L11 Cloud Native Applications

Monolithic architecture. It is the **traditional way of building applications**: built as a **single and indivisible unit**. It usually contains a **client-side UI, a server-side application, a database and a middleware**. **Strengths:** fewer cross-cutting concerns (functions like logging, caching and monitoring), easier debugging and testing, simple to deploy, simple to develop. **Weaknesses:** too complex to understand, challenging to make changes, **cannot scale components independently**, difficult to apply new technologies / features.

Microservices architecture. It **breaks down an application into a collection of smaller independent units (microservices)**. These units carry out every application process as a **separate service**, are built as **independently deployable modules**, and communicate with each other through **pre-defined methods / APIs**. Each service can be **updated and scaled independently**. **Strengths:** independent components, easier understanding, better scalability, flexibility in choosing the technology, higher level of agility. **Weaknesses:** extra complexity of a distributed system, cross-cutting concerns, harder to debug and test.

Monolithic vs microservices scaling. A monolithic app contains domain specific functionality and is normally divided by functional layers such as web, business and data; it **scales by cloning the whole app on multiple servers / VMs / containers**. A microservice application separates functionality into separate smaller services and **scales out by deploying each service independently**, creating instances of those services across servers, VMs or containers.

Cloud native computing. It is a **modern approach of building applications that exploits the flexibility, scalability and resilience of cloud computing**. It was pioneered by companies commonly referred to as **"born in the cloud"** (Netflix, Spotify, Uber, Airbnb). The **Cloud Native Computing Foundation (CNCF)** focuses on making cloud native computing available everywhere and defines it more narrowly, focusing on **containerization**: cloud-native technologies empower organizations to build and run **scalable applications in modern, dynamic environments such as public, private and hybrid clouds**.

12 factor app. It is a **methodology or a set of principles for building scalable, performant, independent, resilient enterprise applications**. It aligns with the principles of microservice architecture and was introduced by developers at **Heroku**. The twelve factors: **1 Codebase:** exactly one codebase for a deployed service, used for many deployments. **2 Dependencies:** all dependencies declared, with no implicit reliance on system tools or libraries. **3 Config:** configuration that varies between deployments is stored in the environment. **4 Backing services:** treated as attached resources, attached and detached by the execution environment. **5 Build, release, run:** the delivery pipeline strictly consists of these three stages. **6 Processes:** deployed as one or more stateless processes, with persisted data stored on a backing service. **7 Port binding:** self-contained services make themselves available to other services by specified ports. **8 Concurrency:** advocated by scaling individual processes. **9 Disposability:** fast startup and shutdown for a more robust and resilient system. **10 Dev/Prod parity:** all environments as similar as possible. **11 Logs:** produced as event streams, leaving the execution environment to aggregate them. **12 Admin processes:** any needed admin tasks are kept in source control and packaged with the application.

Cloud native tools. Cloud delivery model: the way of provisioning, consuming and managing resources, as a public or private cloud solution. **Containers:** make applications more manageable and can provide innovative solutions around design, scaling, security and resiliency. **Microservices:** a collection of services, each performing a specific business function. **Service mesh:** manages complex service-to-service communication, scalable, secure, fast and reliable. **Orchestration:** manages provisioning, deploying, operating and scaling of containerized microservices.

Cloud native practices. DevOps (Development and Operations), **SRE** (Site Reliability Engineering), **NoOps** (No IT Operations), **CI** (Continuous Integration) and **CD** (Continuous Delivery and Deployment).

SRE and NoOps. SRE: Google defined a model called Site Reliability Engineering as a **collection of best practices for any digital business** that takes contemporary circumstances in the highly competitive market seriously. **NoOps:** DevOps practitioners are **no longer required to take care of operations** and may concentrate on software development. It can be achieved as **everything that could be automated is already automated**.

Continuous integration (CI). The practice of **automating the integration of code changes from multiple contributors into a single software project**. It allows developers to **frequently merge code changes into a central repository**, where builds and tests then run. Automated tools are used to **assert the new code's correctness before integration**.

Continuous delivery and deployment (CD). CD is a software release process that uses **automated testing to validate that changes to a codebase are correct and stable** for immediate autonomous deployment to a production environment. It allows engineering organizations to focus on core business needs instead of infrastructure overhead. **Continuous delivery:** developers review and merge code changes, which are packaged into an **artifact**, then moved to a production environment where it **awaits approval** before deployment. **Continuous deployment:** the package is opened and reviewed with a system of **automated checks**; if the checks fail the package is rejected, and when they pass it is **automatically deployed to production**.

CI/CD tools and services. Tools: Jenkins, GitLab, GitHub, Bitbucket, CircleCI, TeamCity, Bamboo, Travis CI. **Azure DevOps Services:** Azure Boards, Azure Pipelines, Azure Repos, Azure Test Plans, Azure Artifacts. **AWS DevOps Services:** CodePipeline, CodeBuild, CodeDeploy, CodeStar. **Google DevOps Services:** Cloud Build, Artifact Registry, Binary Authorization, Tekton, Spinnaker, Operations Suite.

Application modernization. It is the **refactoring, re-purposing or consolidation of legacy software programming to align it more closely with current business needs**. The goal is to **create new business value from existing applications**.

Application modernization journey. Five steps: **1 Traditional app**; **2 existing app hosted as a container or executable** (rehost); **3 existing application plus new microservices** (refactor); **4 parts of the existing application re-architected** (rearchitect); **5 transformed microservices application** (rearchitect / rebuild).

L12 Containerization and Docker

Why containers? The matrix from hell. An application depends on the whole stack: **hardware infrastructure, operating system, libraries and dependencies**. The same application must run in many environments (**development, testing, QA, staging and production**), so there is a requirement to maintain different environments, and every application and environment combination has to be made to work. This is called **the matrix from hell**. The solution is a **container platform** on top of one operating system, where each application keeps its own libraries and dependencies.

Containers. A container is a **lightweight, self-contained software package that bundles an application's code with all its dependencies**, such as libraries and system settings, into a single unit. This packaging ensures the application **runs consistently across different computing environments**, like a developer's laptop or a cloud server, because it includes everything it needs to run, without relying on the host's operating system. Each container has its own **network, mounts and processes**, and all containers run on the same **operating system kernel** through the container platform.

Virtualization vs containerization. Virtualization: hardware infrastructure, a host operating system, a **hypervisor**, and on top of it virtual machines that each hold an application, its libs and deps and a **full guest OS**. **Containerization:** hardware infrastructure, an operating system, a **container engine**, and on top of it containers that each hold only the application, libs and deps with **no guest OS**. So containers are lighter, start faster and use fewer resources.

Docker. Docker is a tool that allows developers and sys-admins to easily **deploy their applications in a sandbox (called containers)** to run on the host operating system, that is Linux. It is written in the **Go programming language** and takes advantage of several features of the Linux kernel. Docker uses **namespaces** to provide the isolated workspace called the container: when you run a container, Docker creates a set of namespaces for it, each aspect of the container runs in a separate namespace and its access is limited to that namespace.

Docker components. Software: Docker daemon (dockerd) and Docker client (docker). **Objects:** container and image. Plus the **registry**. The **Docker daemon (dockerd)** listens for Docker API requests and manages Docker objects such as images, containers, networks and volumes, and can communicate with other daemons to manage Docker services. The **Docker client (docker)** is the primary way users interact with Docker: when you use commands such as docker run, the client sends them to dockerd, which carries them out, using the Docker API. The client can communicate with more than one daemon.

Docker Desktop. An easy-to-install application for Mac, Windows or Linux that lets you build and share containerized applications and microservices. It includes the **Docker daemon, Docker client, Docker Compose, Docker Content Trust, Kubernetes and Credential Helper**.

Docker image. An image is a **read-only template with instructions for creating a Docker container**. Often an image is based on another image with some additional customization, for example an image based on ubuntu that installs the Apache web server and your application and the configuration to run it. The image defines how a container should be created and which software components will run, and once an image is run it becomes a container. **Image = blueprint (static, read-only). Container = running instance of that image (dynamic, executable).**

Dockerfile. A Dockerfile is a **simple text file that contains a list of commands that the Docker client calls while creating an image**. It is a simple way to **automate the image creation process**, and the commands are almost identical to their equivalent Linux commands.

Docker container. A container is a **runnable instance of an image**. It packages the application code with all its dependencies and runs it in an isolated environment. Example: an image of Ubuntu OS with an NGINX server, run with docker run, creates a container with NGINX running on Ubuntu. A container is created when an image is executed, runs as an **isolated process on the host machine but shares the host's operating system kernel**, and many containers can run on the same system without interfering with each other. You can create, start, stop, move or delete a container using the Docker API or CLI, connect it to networks, attach storage, or create a new image from its current state.

Docker registry and Docker Hub. A registry is a **centralized storage and distribution system for Docker images**, a repository where users store, manage and share container images. **Docker Hub** is the default and most widely used **public registry**, Docker looks for images there by default, and you can also run your own **private registry**. Cloud registries: **Amazon Elastic Container Registry (ECR), Azure Container Registry and Google Container Registry**. Docker Hub is a cloud-based service where people **push and pull** container images over the internet, publicly or privately, mainly used by DevOps teams. It is open source and free for all operating systems.

Docker architecture and working. Docker uses a **client-server architecture**. The Docker client talks with the Docker daemon, which builds, runs and distributes containers. They can run on the same system or the client can connect to a remote daemon, and they interact through a **REST API over a UNIX socket or a network**. In the DevOps process, development hands an application plus a guide (the Docker image) to operations.

Advantages of containers. Portability: runs anywhere (local machine, cloud, on-premises servers). **Consistency:** same behavior in development, testing and production. **Lightweight:** no full OS per app, containers share the host kernel. **Scalability:** ideal for microservices and orchestrators like Kubernetes and Docker Swarm. **Efficiency:** starts in seconds and uses fewer system resources.

Docker commands: running containers. docker pull <IMAGE> fetches an image from a registry such as Docker Hub to the local machine (docker images lists them). **docker create** creates a new container from an image without starting it (docker create --name my_hello_world hello-world, then docker ps -a shows it and the output is the container ID). **docker start** restarts a stopped container, resuming from its previous state. **docker run** launches a container from an image with runtime options and commands, and it equals **docker pull + docker create + docker start** (for example docker run hello-world, docker run busybox ls).

Docker commands: managing containers. docker logs views the logs generated by a container. **docker attach** attaches your terminal's standard input, output and error to a running container. **docker exec -it <CONTAINER> <COMMAND>** runs a new command in a running container (docker exec -it ID ping www.sliit.lk), and **docker run -it busybox sh** opens an interactive shell.

Docker commands: stopping containers. docker pause / unpause suspends or resumes all processes in the container (sends **SIGSTOP / SIGCONT**). **docker restart** restarts one or more containers. **docker stop** halts the container by gracefully shutting down its processes (sends **SIGTERM**). **docker kill** kills a running container immediately (sends **SIGKILL**), and docker kill --signal=SIGTERM equals docker stop, SIGSTOP equals pause, SIGCONT equals unpause.

Docker commands: removing. docker rm <CONTAINER> removes one or more containers. **docker system prune** removes all stopped containers (and other unused objects). **docker rmi <IMAGE>** removes (and untags) one or more images from the host node.

Container orchestration. It is the process of **automating the deployment, management, scaling and networking of containers throughout their lifecycle**, making it possible to deploy software consistently across many environments at scale. It is necessary to manage the complexity of modern, large-scale containerized applications by automating deployment, scaling and load balancing, which improves agility, resilience and efficiency, reduces manual effort and errors, and allows more efficient resource use, leading to cost savings and faster development cycles.

Why containers need orchestration. Containers are smaller, more resource-efficient and more portable than VMs, and containerized microservices or serverless functions have become the **de facto compute units of modern cloud-native applications**. In small numbers containers are easy to manage manually, but at scale it is not practical. A **CI/CD or DevOps pipeline is impossible without container orchestration**, which automates the operational tasks of deploying and running containerized applications and services.

How container orchestration works. The architecture consists of **container clusters running across multiple machines and environments**. Each cluster is a group of **nodes** (server instances). **Worker nodes run containers** using container runtimes such as Docker, and a group of **control plane nodes act as the orchestrator** of the cluster. Users manage and monitor workloads through tools with APIs and GUIs. It is essentially a three-step cycle: **provisioning, deployment and lifecycle management**. Tools: **Kubernetes and Docker Swarm**.

Advantages of container orchestration. Automation and simplified operations: automates complex tasks such as deploying, scaling and managing containers, increases speed and agility, and simplifies large-scale management through a declarative way to automate operations. **Improved reliability and resilience:** automatic health checks and self-healing reduce downtime, and service continuity keeps the application operational even if a container fails. **Resource efficiency and cost savings:** containers are scheduled on the most efficient hardware, and applications scale up or down automatically so you pay only for what you use. **Better security and portability:** reduces human error and isolates applications from the host and each other, and applications run consistently on-premises or on various public clouds, preventing vendor lock-in. **Support for modern development practices:** a natural fit for DevOps and CI/CD pipelines, and ideal for microservices because orchestration makes managing many microservice containers practical.

L10b Infrastructure as Code (IaC)

Risks in manual processes. Building an architecture by hand in the management console (configure data storage, route your network, create your instances, build your databases) is a long manual process. Risks: it **does not support repeatability at scale** (how will you replicate deployments to multiple Regions?), there is **no version control** (how will you roll back production to a prior version?), there is a **lack of audit trails** (how will you ensure compliance and track changes to configuration details at the resource level?) and **inconsistent data management** (how will you ensure matching configurations across multiple EC2 instances?).

What is Infrastructure as Code (IaC). IaC is an industry term for the process of **provisioning and managing cloud resources by defining them in a template file that is both human-readable and machine-consumable**. It enables you to **define, provision and manage your infrastructure using code instead of manual configurations**. Configurations traditionally handled through the AWS console are now maintained as code in your repository, which streamlines deployments and ensures consistency across environments.

Without IaC vs with IaC. Without IaC: manual setup phases for each deployment; teardowns and deployments are slow; setups (hardware and software) are prone to errors; best practices are documented separately; troubleshooting and manual work is required for rollbacks. With IaC: **code is used to specify configurations and is reused**; teardowns and deployments are **instant**; setups are **stored in version control**; best practices are **written inside the code**; rollbacks are **simple and instant**.

Types of IaC tools. Four types: **configuration management tools, server templating tools, orchestration tools and provisioning tools**. **Configuration management tools** focus on installing and managing software on existing servers and keep all servers consistently configured: **Puppet, Chef, Ansible and SaltStack**. **Server templating tools** create reusable templates that define a server's configuration, so new servers are identically configured to a defined standard: **Docker, Vagrant and Packer**. **Orchestration tools** manage the interconnections and interactions between different parts of an infrastructure, crucial for complex environments with many connected components: **Kubernetes, Docker Swarm and Red Hat OpenShift**. **Provisioning tools** focus on the initial setup of servers and other infrastructure components, making it faster and more reliable: **Terraform, AWS CloudFormation and Google Cloud Deployment Manager**.

How IaC works. IaC works on two approaches. The **imperative approach** specifies **step-by-step commands**: you write explicit, sequential commands that the system executes in a precise order to reach the final setup. The **declarative approach** defines the **desired end state**: you describe the final target configuration and the tool figures out how to build or modify the environment to match it.

IaC benefits. Reduce multiple matching environments: one template can create many identical stacks (for example each with a load balancer and an Auto Scaling group). Rapid deployment of complex environments, **configuration consistency**, simple clean up (deleting the stack deletes the resources it created), and easy propagation of a change to all stacks (modify the template, run update stack on all stacks). Also **reusability, repeatability and maintainability**.

Key benefits of IaC. Consistency and reproducibility: deploy the same configuration across multiple environments reliably, eliminating discrepancies, hidden settings and manual errors. **Automation and speed:** create, update or destroy entire environments in minutes by executing a single script, which drastically reduces deployment time. **Version control:** use tools like Git to track changes, review history and collaborate on infrastructure code just as with application source code.

IaC best practices. Use little or no documentation: define specifications and parameters in the configuration files, because additional documentation that gets out of step with the configurations in use is not required. **Version control all configuration files:** place them under source control, which gives flexibility and accountability and lets previous manifests be monitored, controlled and restored. **Constantly test configurations:** test before moving changes to production, and set up automated tests that run when the configuration code is altered. **Modular approach:** divide the infrastructure into different components and merge them by automation, which gives many benefits and restricts the number of modifications to manifests.

Terraform. HashiCorp Terraform is an IaC tool that lets you define **both cloud and on-prem resources in human-readable configuration files** that you can version, reuse and share. You then use a **consistent workflow to provision and manage all of your infrastructure throughout its lifecycle**. It can manage low-level components (compute, storage, networking) and high-level components (DNS entries and SaaS features).

How Terraform works. Terraform creates and manages resources on cloud platforms and other services through their **application programming interfaces (APIs)**. **Providers** enable Terraform to work with virtually any platform or service with an accessible API. The core workflow has three stages: **Write** (define resources, possibly across multiple cloud providers, for example an application on VMs in a VPC with security groups and a load balancer), **Plan** (Terraform creates an execution plan describing what it will create, update or destroy based on the existing infrastructure and your configuration) and **Apply** (on approval, it performs the operations in the correct order, respecting resource dependencies, for example recreating the VPC before scaling the VMs in it).

AWS CloudFormation. It is a service that helps to **model and set up AWS resources** so you spend less time managing them and more time on your applications. You create a **template that describes all the AWS resources you want** (such as EC2 instances or RDS DB instances), and CloudFormation takes care of **provisioning and configuring** those resources. You do not need to individually create and configure resources and figure out what depends on what, because CloudFormation handles that. A template creates a **stack**, and deleting the stack deletes the resources it created.